(2) Explicit type conversion

C++ allows explicit conversion of variables or expressions from one data type to another specific data type by the programmer. It is called as **"type casting".**

Syntax:

(type-name) expression;

Where type-name is a valid C++ data type to which the conversion is to be performed.

Example:

```
#include <iostream>
using namespace std;
int main( )
{
        float varf=78.685;
        cout << (int) varf;
}
```

In the above program, variable **varf is** declared as a **float** with an initial value 78.685. The value of **varf** is explicitly converted to an **int** type in cout statement. Thus, the final output will be 78.

During explicit conversion, if you assign a value to a type with a greater range, it does not cause any problem. But, assigning a value of a larger type to a smaller type may result in loosing or loss of precision values.

S.No	Explicit Conversion	Problem
1	double to float	Loss of precision. If the original value is out of range for the target type, the result becomes undefined
2	float to int	Loss of fractional part. If original value may be out of range for target type, the result becomes undefined
3	long to short	Loss of data

Table 9.12 – Explicit Conversion Problems

Example:

```
#include <iostream>
using namespace std;
int main()
{
        double varf=178.25255685;
        cout << (float) varf << endl;
        cout << (int) varf << endl;
}
```

Output:

178.253
178

11th Std Computer Science_EM Chapter 9.indd 148 06-01-2023 15:08:46